



W4.3 COS30045

Lecture

Agenda

- Setting up
- Loading data
- Bar Chart

Reference

- Dufour & Meeks (2024) Ch 3

D3 data workflow

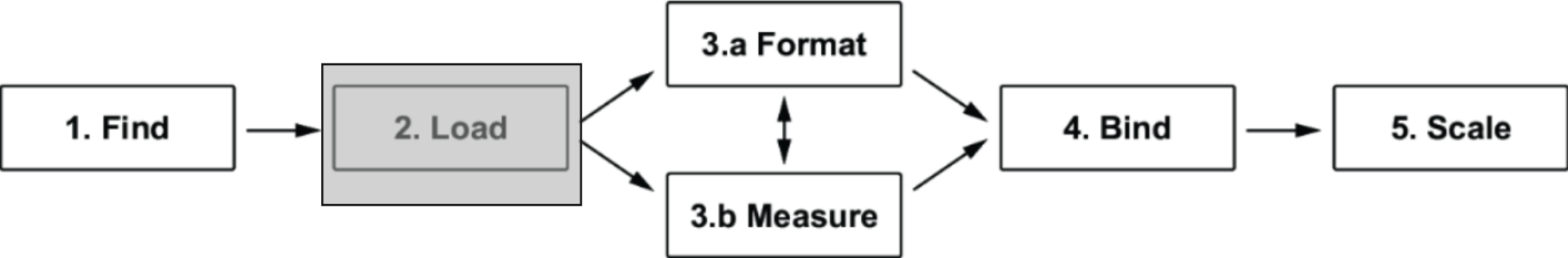


Figure 3.2 Simplified diagram of the D3 data workflow

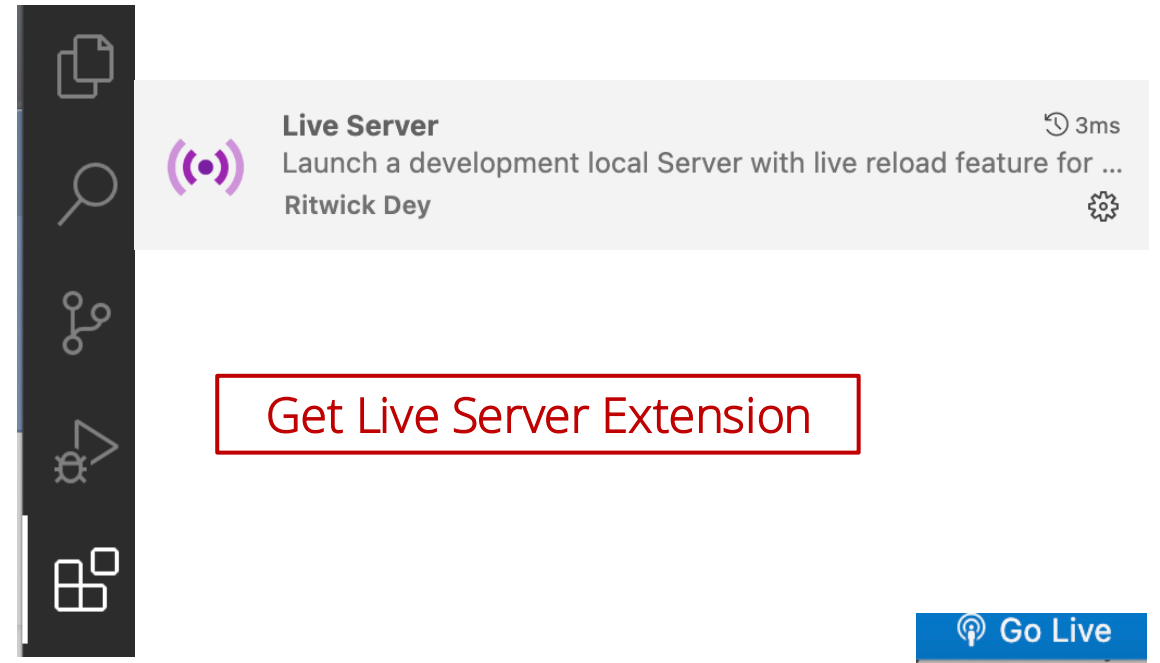
.
.

Loading data into D3

Different load functions for different dataset format

- CSV file -> `d3.csv()`

In VS Code, need to run Live Server to allow it to fetch data



D3 data workflow

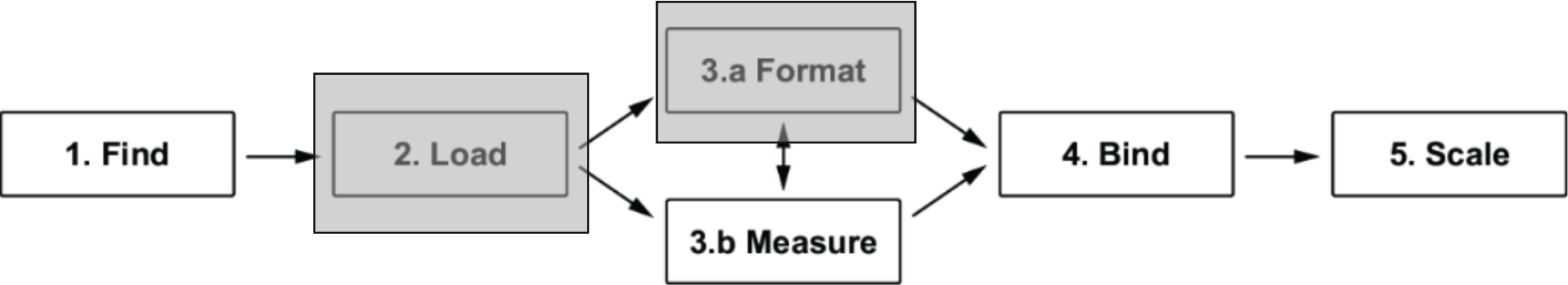


Figure 3.2 Simplified diagram of the D3 data workflow

• • • • • • • • • •
• • • • • • • • • •

Add data from CSV

Data path needs to be specified correctly

Column names must match

Often data fetched as strings

Need to convert to number

```
const svg = d3.select(".responsive-svg-container")
  .append("svg")
  .attr("viewBox", "0 0 1200 1600")
  .style("border", "1px solid black");
```

```
d3.csv("../data/VegetationHab.csv", d => {
  return {
    habitat: d.habitat,
    sighting: +d.sighting
  };
}).then(data => {
```

Arrow function syntax
Shorthand syntax for writing a function

```
d3.csv("../data/VegetationHab.csv", function(d) {
  return {
    habitat: d.habitat,
    sighting: +d.sighting
  };
});
```

```
console.log(data);
```

console

```
▼ Array(8) [ {}, {}, {}, {}, {}, {}, {}, {} ]
  ▶ 0: Object { habitat: "Modified land", sighting: "1673" }
  ▶ 1: Object { habitat: "Dry eucalypt forest and woodland", sighting: "1613" }
  ▶ 2: Object { habitat: "Wet eucalypt forest and woodland", sighting: "723" }
  ▶ 3: Object { habitat: "Native grassland", sighting: "411" }
  ▶ 4: Object { habitat: "Scrub, heathland and coastal complexes", sighting: "156" }
  ▶ 5: Object { habitat: "Highland and treeless vegetation", sighting: "48" }
  ▶ 6: Object { habitat: "Rainforest and related scrub", sighting: "23" }
  ▶ 7: Object { habitat: "Moorland, sedgeland and rushland", sighting: "19" }
```

```
});
```

```
▼ Array(8) [ {}, {}, {}, {}, {}, {}, {}, {} ]
  ▶ 0: Object { habitat: "Native grassland", sighting: 411 }
  ▶ 1: Object { habitat: "Scrub, heathland and coastal complexes", sighting: 156 }
  ▶ 2: Object { habitat: "Dry eucalypt forest and woodland", sighting: 1613 }
  ▶ 3: Object { habitat: "Highland and treeless vegetation", sighting: 48 }
  ▶ 4: Object { habitat: "Modified land", sighting: 1673 }
  ▶ 5: Object { habitat: "Rainforest and related scrub", sighting: 23 }
  ▶ 6: Object { habitat: "Wet eucalypt forest and woodland", sighting: 723 }
  ▶ 7: Object { habitat: "Moorland, sedgeland and rushland", sighting: 19 }
```

D3 data workflow

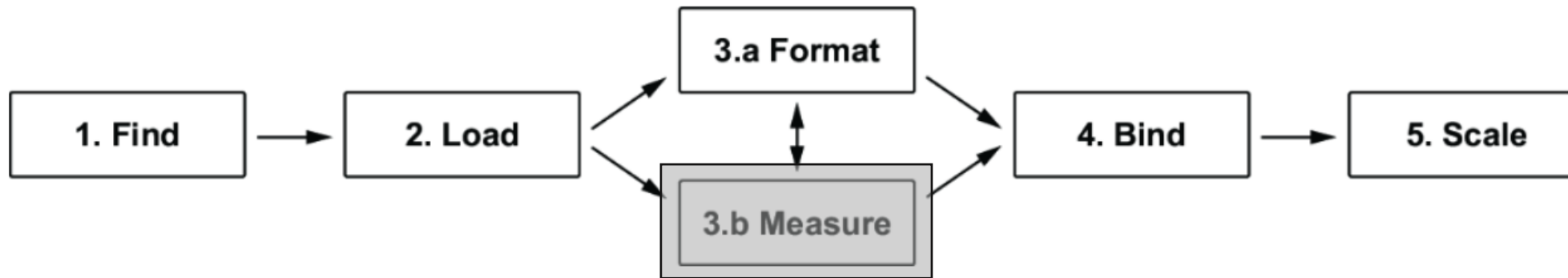


Figure 3.2 Simplified diagram of the D3 data workflow

• • • • • • • • • •
• • • • • • • • • •

Access information about dataset

```
d3.csv("../data/VegetationHab.csv", d => {  
  return {  
    habitat: d.habitat,  
    sighting: +d.sighting  
  };  
}
```

```
}).then(data => {
```

```
  console.log(data);
```

```
  console.log(data.length);
```

```
  console.log(d3.max(data, d => d.sighting));
```

```
  console.log(d3.min(data, d => d.sighting));
```

```
});
```

```
▼ Array(8) [ {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...} ]  
  ▶ 0: Object { habitat: "Native grassland", sighting: 411 }  
  ▶ 1: Object { habitat: "Scrub, heathland and coastal complexes", sighting: 156 }  
  ▶ 2: Object { habitat: "Dry eucalypt forest and woodland", sighting: 1613 }  
  ▶ 3: Object { habitat: "Highland and treeless vegetation", sighting: 48 }  
  ▶ 4: Object { habitat: "Modified land", sighting: 1673 }  
  ▶ 5: Object { habitat: "Rainforest and related scrub", sighting: 23 }  
  ▶ 6: Object { habitat: "Wet eucalypt forest and woodland", sighting: 723 }  
  ▶ 7: Object { habitat: "Moorland, sedgeland and rushland", sighting: 19 }  
  ▶ columns: Array [ "habitat", "sighting" ]  
    length: 8  
  ▶ <prototype>: Array []
```

8

1673

19

.
.

Access information about dataset

```
d3.csv("../data/VegetationHab.csv", d => {  
  return {  
    habitat: d.habitat,  
    sighting: +d.sighting  
  };  
  
}).then(data => {  
  console.log(data);  
  console.log(data.length);  
  console.log(d3.max(data, d => d.sighting));  
  console.log(d3.min(data, d => d.sighting));  
  
  data.sort((a, b) => b.sighting - a.sighting);  
  
});
```

```
▼ Array(8) [ {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...} ]  
  ▶ 0: Object { habitat: "Native grassland", sighting: 411 }  
  ▶ 1: Object { habitat: "Scrub, heathland and coastal complexes", sighting: 156 }  
  ▶ 2: Object { habitat: "Dry eucalypt forest and woodland", sighting: 1613 }  
  ▶ 3: Object { habitat: "Highland and treeless vegetation", sighting: 48 }  
  ▶ 4: Object { habitat: "Modified land", sighting: 1673 }  
  ▶ 5: Object { habitat: "Rainforest and related scrub", sighting: 23 }  
  ▶ 6: Object { habitat: "Wet eucalypt forest and woodland", sighting: 723 }  
  ▶ 7: Object { habitat: "Moorland, sedgeland and rushland", sighting: 19 }  
  ▶ columns: Array [ "habitat", "sighting" ]  
  ▶ length: 8  
  ▶ <prototype>: Array []
```

8

1673

19

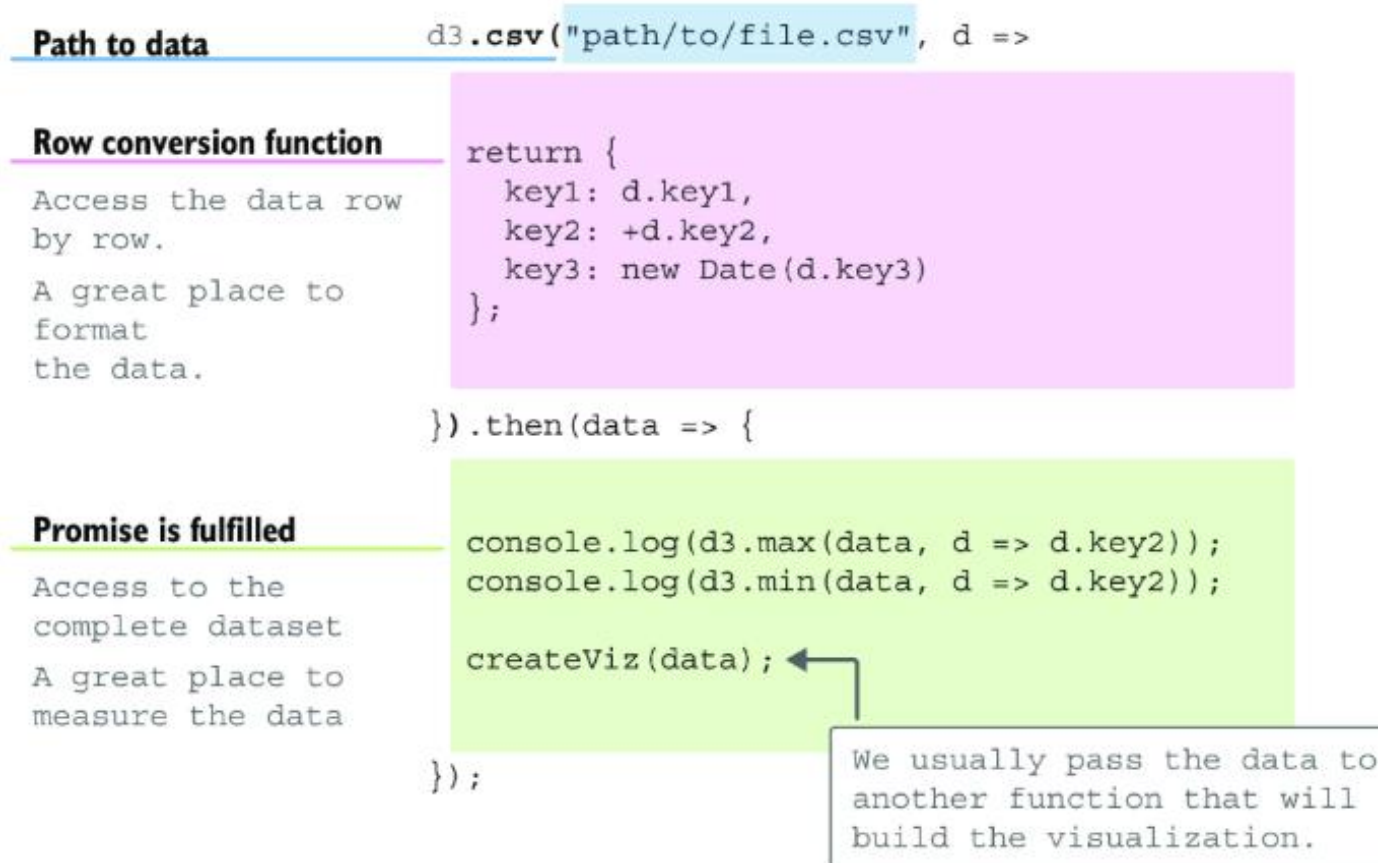
```
▼ Array(8) [ {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...} ]  
  ▶ 0: Object { habitat: "Modified land", sighting: 1673 }  
  ▶ 1: Object { habitat: "Dry eucalypt forest and woodland", sighting: 1613 }  
  ▶ 2: Object { habitat: "Wet eucalypt forest and woodland", sighting: 723 }  
  ▶ 3: Object { habitat: "Native grassland", sighting: 411 }  
  ▶ 4: Object { habitat: "Scrub, heathland and coastal complexes", sighting: 156 }  
  ▶ 5: Object { habitat: "Highland and treeless vegetation", sighting: 48 }  
  ▶ 6: Object { habitat: "Rainforest and related scrub", sighting: 23 }  
  ▶ 7: Object { habitat: "Moorland, sedgeland and rushland", sighting: 19 }  
  ▶ columns: Array [ "habitat", "sighting" ]  
  ▶ length: 8  
  ▶ <prototype>: Array []
```

8

1673

19

Pattern for loading and transforming data in D3



.
.

Figure 3.14 How and where to load, transform, and measure data in D3

Set up function to create chart and pass it the dataset

```
d3.csv("../data/VegetationHab.csv", d => {  
  return {  
    habitat: d.habitat,  
    sighting: +d.sighting  
  };  
  
}).then(data => {  
  console.log(data);  
  console.log(data.length);  
  console.log(d3.max(data, d => d.sighting));  
  console.log(d3.min(data, d => d.sighting));  
  
  data.sort((a, b) => b.sighting - a.sighting);  
  
  drawBarChart(data);  
  
});
```

.
.

D3 data workflow

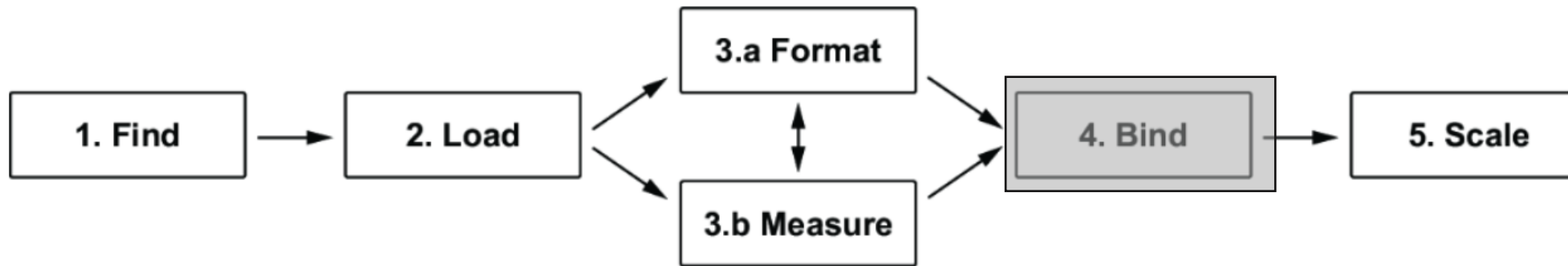


Figure 3.2 Simplified diagram of the D3 data workflow

• • • • • • • • • •
• • • • • • • • • •

Binding Data to the DOM

Pattern for binding data

selection

`.selectAll("selector")`

`.data(myData)`

`.join("element to add");`

Note 'Old' binding pattern

selection `.selectAll("selector")`

`.data(myData)`

`.enter()`

`.append("element type");`

Use join instead...

```
const svg = d3.select(".responsive-svg-container")
  .append("svg")
  .attr("viewBox", "0 0 1200 1600")
  .style("border", "1px solid black");
```

svg

`.selectAll("rect")`

*css selector for element we want to add
(currently this is an empty selection)*

svg

`.selectAll("rect")`

`.data(data)`

*Pass data to svg object (tells d3 to
create one rect for each row in data)*

svg

`.selectAll("rect")`

`.data(data)`

`.join("rect")`

Join method adds rectangles to DOM

```
▼ <div class="responsive-svg-container"> (overflow)
  <h1>Wombat populations in Tasmania</h1>
  ▼ <svg viewBox="0 0 1200 1600" style="border: 1px solid black;">
    <rect></rect>
    <rect></rect>
    <rect></rect>
    <rect></rect>
    <rect></rect>
    <rect></rect>
    <rect></rect>
```

Overview of binding process

- Select the svg canvas
- Create an empty selection and give it something to select (e.g., rect)
- Pass the dataset to the method
- For each row of the dataset create a rect
- Add rect to DOM

Now we can start drawing the rect according to data in the dataset!

1. Selection

```
<div class="responsive-svg-container">  
  <svg viewBox="0 0 1200 1600" style="border: 1px solid black;"></svg> == $0  
</div>
```

2. Data binding (in the code editor)

```
const myData = [  
  { technology: "Excel", count: 1078 },  
  { technology: "Tableau", count: 852 },  
  { technology: "PowerPoint", count: 681 },  
  { technology: "R", count: 561 },  
  { technology: "Python", count: 530 }  
];  
  
d3.select("svg")  
  .selectAll("rect")  
  .data(myData)  
  .join("rect");
```

3. DOM with data-bound elements

```
<div class="responsive-svg-container">  
  <svg viewBox="0 0 1200 1600">  
    <rect></rect>  
    <rect></rect>  
    <rect></rect>  
    <rect></rect>  
    <rect></rect>  
  </svg>
```

Build bar chart function

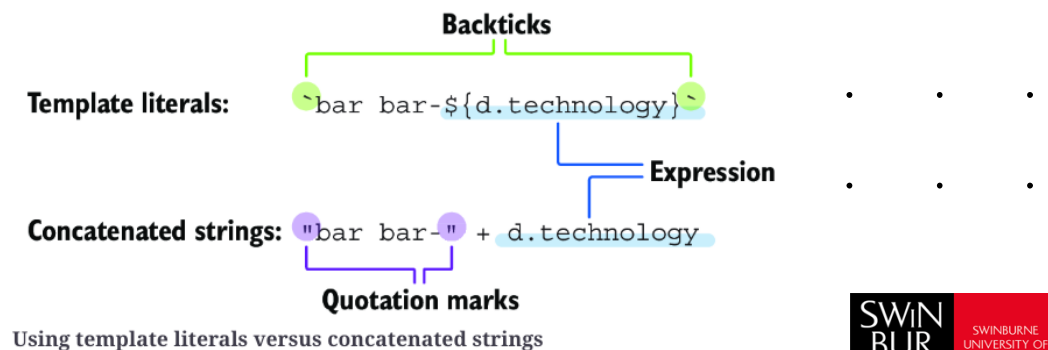
```
const svg = d3.select(".responsive-svg-container")
  .append("svg")
  .attr("viewBox", "0 0 1200 1600")
  .style("border", "1px solid black");
```

```
const drawBarChart = data => {
  ...
  svg
    .selectAll("rect")
    .data(data)
    .join("rect")
    .attr("class", d => {
      console.log(d);
      return `bar bar-${d.sighting}`;
    })
};
```

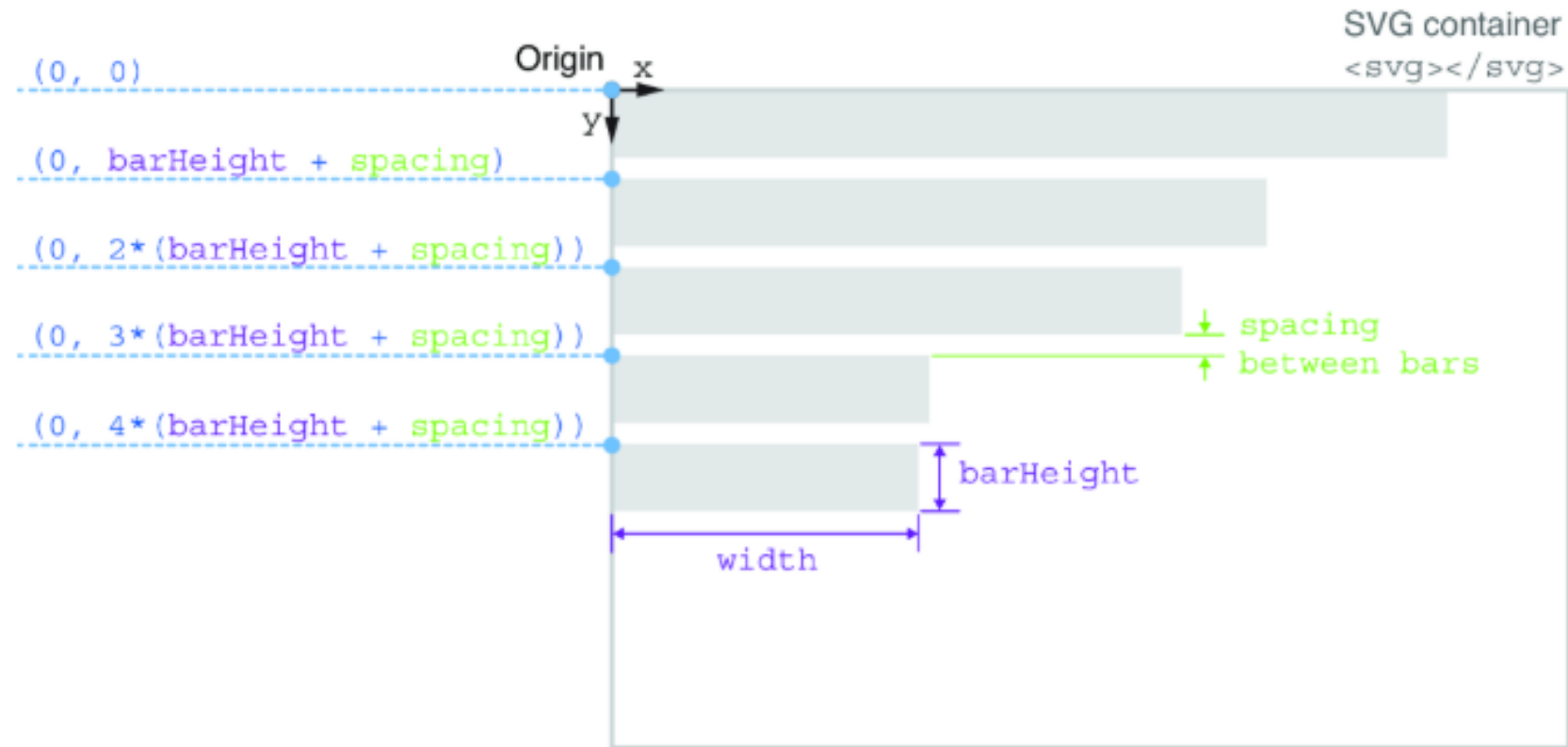
```
<svg viewBox="0 0 1200 1600" style="border: 1px solid black;">
  <rect class="bar bar-1673"></rect>
  <rect class="bar bar-1613"></rect>
  <rect class="bar bar-723"></rect>
  <rect class="bar bar-411"></rect>
  <rect class="bar bar-156"></rect>
  <rect class="bar bar-48"></rect>
  <rect class="bar bar-23"></rect>
  <rect class="bar bar-19"></rect>
</svg>
```

Set a class attribute to our bars (so we can more easily control their style later)

Dynamically generate class name for rectangles from data set using template literal



Working out how to layout bar chart



Bar height and spacing will be constant, bar width must reflect data

Figure 3.18 Finding a formula for the position of the top-left corner of each rectangle

Working out bar chart layout

```
const barHeight = 20;
```

```
svg
```

```
.selectAll("rect")  
.data(data)  
.join("rect")  
.attr("class", d => {  
  console.log(d);  
  return `bar bar-${d.sighting}`;  
})  
.attr("width", d => d.sighting)  
.attr("height", barHeight)  
};
```

```
<h1>Wombat populations in Tasmania</h1>  
▼ <svg viewBox="0 0 1200 1600" style="border: 1px solid black;">  
  <rect class="bar bar-1673" width="1673" height="20"></rect>  
  <rect class="bar bar-1613" width="1613" height="20"></rect>  
  <rect class="bar bar-723" width="723" height="20"></rect>  
  <rect class="bar bar-411" width="411" height="20"></rect>  
  <rect class="bar bar-156" width="156" height="20"></rect>  
  <rect class="bar bar-48" width="48" height="20"></rect>  
  <rect class="bar bar-23" width="23" height="20"></rect>  
  <rect class="bar bar-19" width="19" height="20"></rect>  
</svg>
```

Wombat populations in Tasmania

All bars laid on top of each other!

Width assigned sightings attribute

Height assigned const `barHeight`

Spreading out the bars

x will always equal 0

y will be function of

- position in data set plus
- **barHeight** and
- spacing

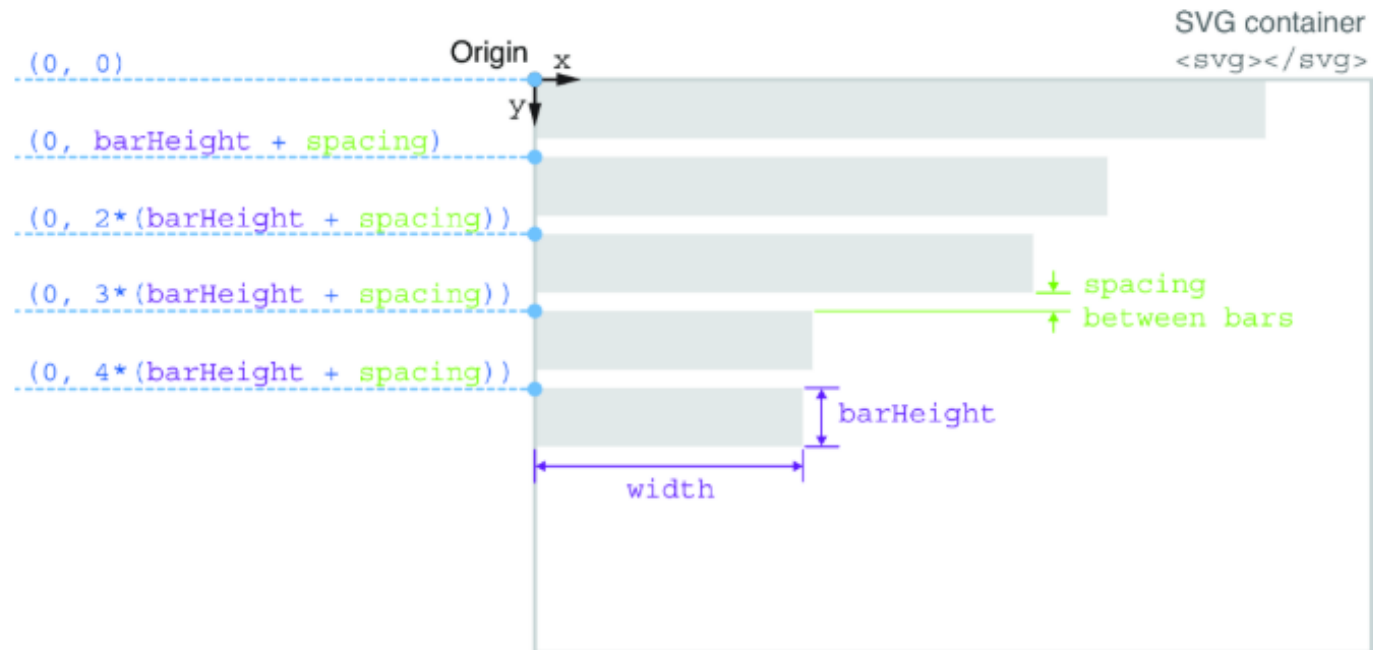


Figure 3.18 Finding a formula for the position of the top-left corner of each rectangle

```
const barHeight = 20;
```

```
svg
.selectAll("rect")
.data(data)
.join("rect")
  .attr("class", d => {
    console.log(d);
    return `bar bar-${d.sighting}`;
  })
  .attr("width", d => d.sighting)
  .attr("height", barHeight)
  .attr("x", 0)
  .attr("y", (d, i) => (barHeight + 5) * i);
```

Wombat populations in Tasmania



• • • • • • • • • •

• • • • • • • • • •

Syntax rules for arrow functions

```
selection
```

```
.attr("width", d => d.count) Arrow function is displayed on one line.
```

```
.attr("y", (d, i) => (barHeight + 5) * i)
```

If more than one parameter, we wrap them in parentheses.

```
.attr("class", d => {  
  console.log(d);  
  return "bar";  
})
```

If the function is spread over multiple lines,
we use body braces
and a return statement.

Figure 3.19 Formatting arrow functions

JavaScript ternary operator

```
.attr("fill", d => d.habitat === "Rainforest and related scrub" ? "blue" : "green");
```

```
d => d.habitat === "Rainforest and related scrub" ? "blue" : "green");
```

condition

If true

If false

Wombat populations in Tasmania



Make all green except item of interest

.
.

D3 data workflow

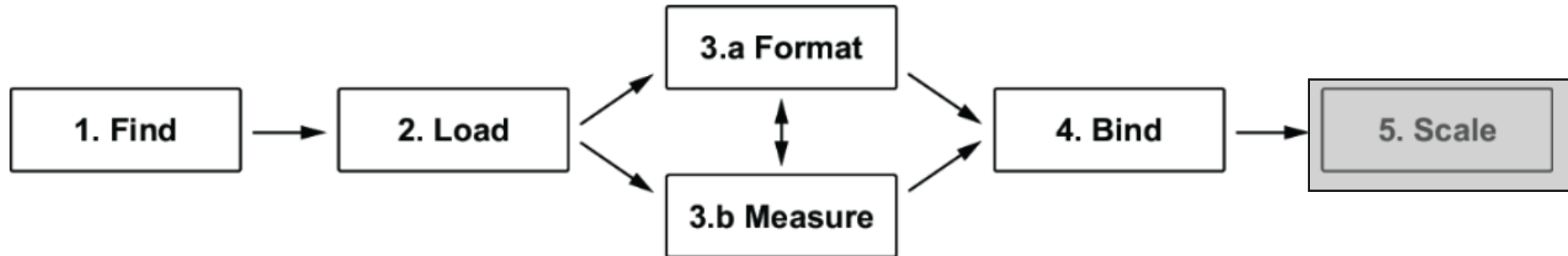


Figure 3.2 Simplified diagram of the D3 data workflow

.
.